

DMIT2015 – Lesson 6 Student Practice Activity

JSF Navigation + Lifecycle

Overview

In this activity, you will build a multi-page Jakarta Faces (JSF) course enrollment workflow.

You will:

- create a form page
- validate user input
- navigate to a confirmation page
- use redirect navigation
- observe JSF lifecycle behavior

Unlike the previous lesson, this activity requires more independent problem solving and adapting existing patterns.

Learning Outcomes

By completing this activity, you should be able to:

- Navigate between JSF pages
- Use action methods for navigation
- Use `faces-redirect=true`
- Explain how validation affects navigation
- Use bean scope appropriately
- Debug common JSF lifecycle issues

Estimated Time

30–35 minutes

Application Requirements

You will create:

File	Purpose
course-enrollment.xhtml	Enrollment form page
enrollment-confirmation.xhtml	Confirmation page
EnrollmentBean.java	Managed bean

Part 1 — Create the Managed Bean

Step 1 — Create Package

Create:

```
dmit2015.view
```

Step 2 — Create EnrollmentBean.java

Create:

```
EnrollmentBean.java
```

Step 3 — Add Required Imports

```
import jakarta.enterprise.context.SessionScoped;  
import jakarta.inject.Named;
```

```
import java.io.Serializable;
```

Step 4 — Create the Bean

Copy and complete the following code:

```
package dmit2015.view;
```

```
import jakarta.enterprise.context.SessionScoped;  
import jakarta.inject.Named;
```

```
import java.io.Serializable;
```

```
@Named
@SessionScoped
public class EnrollmentBean implements Serializable {

    private String studentName;
    private String programName;
    private boolean onlineDelivery;

    public String submit() {

        return "enrollment-confirmation?faces-redirect=true";
    }

    public String getDeliveryMode() {

        if (onlineDelivery) {
            return "Online";
        }

        return "In-Person";
    }

    // Generate getters/setters
}
```

Important Requirement

You MUST generate:

- getters
- setters

for:

- studentName
- programName
- onlineDelivery

Part 2 — Create the Enrollment Form Page

Step 1 — Create the File

Create:

src/main/webapp/course-enrollment.xhtml

Step 2 — Create the XHTML Page

Copy the following starter code:

```
<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core"
      xmlns:p="primefaces">

  <h:head>
    <title>Course Enrollment</title>
  </h:head>

  <h:body>

    <h2>DMIT2015 Course Enrollment</h2>

    <h:form>

      <p:messages />

      <p:panel header="Enrollment Form">

        <!-- Student Name -->

        <p:outputLabel
          for="studentName"
          value="Student Name:" />

        <br/>

        <p:inputText
          id="studentName"
          value="#{enrollmentBean.studentName}"
          required="true"
          requiredMessage="Student name is required" />

        <br/><br/>

        <!-- Program Dropdown -->

        <p:outputLabel
          for="programName"
          value="Program:" />

        <br/>

        <p:selectOneMenu
          id="programName"
```

```

        value="#{enrollmentBean.programName}"
        required="true"
        requiredMessage="Program selection is required">

        <f:selectItem
            itemLabel="Select Program"
            itemValue="" />

        <f:selectItem
            itemLabel="DMIT"
            itemValue="DMIT" />

        <f:selectItem
            itemLabel="CNT"
            itemValue="CNT" />

        <f:selectItem
            itemLabel="NET"
            itemValue="NET" />

    </p:selectOneMenu>

    <br/><br/>

    <!-- Delivery Mode -->

    <p:selectBooleanCheckbox
        value="#{enrollmentBean.onlineDelivery}" />

    <h:outputText value="Online Delivery" />

    <br/><br/>

    <!-- Submit Button -->

    <p:commandButton
        value="Submit Enrollment"
        action="#{enrollmentBean.submit}" />

</p:panel>

</h:form>

</h:body>

</html>

```

Part 3 — Create the Confirmation Page

Step 1 — Create the File

Create:

```
src/main/webapp/enrollment-confirmation.xhtml
```

Step 2 — Create the XHTML Page

Create the confirmation page yourself using:

- `<h:outputText>`
- `<h2>`
- EL expressions

Your page MUST display:

- student name
- selected program
- delivery mode

Expected example output:

Enrollment Successful

Student: Sam Wu

Program: DMIT

Delivery Mode: Online

Part 4 — Run and Test the Application

Step 1 — Start Apache Tomcat

Deploy the application.

Step 2 — Open Browser

Navigate to:

<http://localhost:8080/your-app-name/course-enrollment.xhtml>

Part 5 — Required Tests

Test 1 — Missing Student Name

Leave:

Student Name

blank.

Click:

Submit Enrollment

Expected:

- validation message displays
- navigation does NOT occur

Test 2 – Missing Program Selection

Do not select a program.

Expected:

- validation message displays
- action method does not navigate

Test 3 – Successful Enrollment

Use:

Field	Value
Student Name	Sam Wu
Program	DMIT
Online Delivery	Checked

Expected:

- navigate to confirmation page
- URL changes to:

enrollment-confirmation.xhtml

Part 6 – Observe Lifecycle Behavior

Important Observation

When validation fails:

- the confirmation page does NOT load
- the action method does NOT navigate

This occurs because:

validation happens BEFORE action methods execute

Common Troubleshooting

Problem

Button click appears to do nothing.

Check:

- validation messages
- required fields
- browser console
- Tomcat deployment status

Problem

Confirmation page displays blank values.

Check:

- getters/setters exist
- EL expressions match bean property names
- bean scope is correct

Problem

Page navigation fails.

Check:

```
return "enrollment-confirmation?faces-redirect=true";
```

Problem

HTTP 404 Error

Check:

- file names match exactly
- .xhtml extension correct
- URL typed correctly

Bonus Challenge (Optional)

Challenge 1 — Add Back Button

Add a button on:

`enrollment-confirmation.xhtml`

that navigates back to:

`course-enrollment.xhtml`

Use:

`<p:button>`

OR

`<h:link>`

Challenge 2 — Add Email Address Validation

Add:

`emailAddress`

to the bean.

Requirements:

- input field
- required validation
- display on confirmation page

Challenge 3 — Add Clear Form Logic

Create:

`clearForm()`

method that resets:

- `studentName`
- `programName`
- `onlineDelivery`

Add:

`Clear Form`

button to the enrollment page.

Challenge 4 — Experiment with Bean Scope

Change:

`@SessionScoped`

to:

@RequestScoped

OR

@ViewScoped

Observe:

- what changes
- what breaks
- how redirect affects state

Be prepared to discuss your observations.

Submission

No Brightspace submission required unless instructed by your instructor.

Be prepared to:

- demonstrate successful navigation
- explain redirect behavior
- explain validation flow
- discuss lifecycle observations